



Eskişehir Osmangazi Üniversitesi
Image Processing

2025-2026 Bahar Dönemi

Süleyman Efe Polat - 1521202310
Server Ahıskalı - 152120231118
Eren Çil - 152120231119

Prof. Dr. Kemal Özkan

İçindekiler

Ses Kontrollü Görüntü ve Video İşleme Asistanı - Kullanılan Teknolojiler ve Teknik Tasarım	3
1. Proje Özeti.....	3
2. Kullanılan Teknolojiler.....	3
Yapay Zeka ve Doğal Dil İşleme (AI & NLP).....	3
Görüntü İşleme (Computer Vision & DSP) ve Backend Teknolojileri.....	3
Ses İşleme ve İletişim.....	4
Sistem Mimarisi (Backend & Concurrency).....	4
3. Sistem Mimarisi ve İletişim Protokolleri.....	4
4. Uygulama Modları ve Arayüz.....	4
5. Çalışma Mantığı.....	5
6. Projeden Ekran Görüntüleri.....	5

Ses Kontrollü Görüntü ve Video İşleme Asistanı - Kullanılan Teknolojiler ve Teknik Tasarım

1. Proje Özeti

Bu çalışma; kullanıcıların doğal dil komutlarını gerçek zamanlı olarak analiz eden ve bu komutları karmaşık görüntü işleme algoritmalarına dönüştüren bir "Voice-AI Agent" sistemidir. Proje, yerel büyük dil modellerini dijital sinyal işleme (DSP) teknikleriyle birleştirerek; statik fotoğraflar ve canlı video akışları üzerinde asenkron ve etkileşimli bir düzenleme deneyimi sağlar. Sistem, sestten metne, metinden anlama ve anlamdan görüntü işlemeye uzanan uçtan uca bir yapı sunar.

2. Kullanılan Teknolojiler

Yapay Zeka ve Doğal Dil İşleme (AI & NLP)

- Ollama (Llama 3.1:8b):** Sistemin karar verici beyni gibidir. Kullanıcının serbest konuşma dilini analiz ederek sistemin backend kısmına girdi sağlayan, katı kurallı JSON nesneleri üretir. Özel Sistem Komutu sayesinde eksik parametreleri anlayıp kullanıcıdan ister. Örneğin sistem, şiddet değeri belirtilmeyen bir bulanıklaştırma komutu girilince bunu fark eder ve kullanıcıdan sesli geri bildirim talep eder.
- Faster-Whisper:** Gürültülü ortamlarda dahi yüksek doğrulukla çalışan, VAD (Ses Aktivite Tespiti) destekli Sesten-Metne (STT) çeviri modelidir. VAD (Voice Activity Detection) filtresi kullanılarak ortamdaki parazit sesler ayıklanmış, sadece insan konuşması odaklı bir girdi akışı oluşturulup projeye entegre edilmiştir.

Görüntü İşleme (Computer Vision & DSP) ve Backend Teknolojileri

- OpenCV & NumPy & SciPy:** Görüntü matrisleri üzerindeki tüm ağır matematiksel manipülasyonlar için kullanıldı. Basit filtrelemelerden (Blur, Sharpen) karmaşık algoritmalarına (Sobel/Laplacian Türevleri, Fast Fourier Transform - FFT) kadar tüm görsel çekirdek bu kütüphanelerle yazıldı.
- Fast Fourier Transform (FFT) Operasyonları:** NumPy FFT algoritmaları kullanılarak, görüntülerin frekans uzayına (frequency domain) aktarımı sağlanmıştır. "Color-Preserving FFT" algoritması sayesinde, renkli görüntüler kanallarına (RGB) ayrıştırılarak işlenir ve Ters Fourier (IFFT) ile kayıpsız bir şekilde uzamsal uzaya geri döndürülür.
- Matematiksel Clamping ve Validasyon:** İşlem çekirdeği; LLM tarafından üretilebilecek hatalı veya uç değerlerin (Örn: geçersiz kernel boyutları) sistemi

çökertmesini engelleyen, matematiksel sınırlandırma (clamping) ve tip güvenliği (safe_int) katmanlarıyla korunmaktadır.

Ses İşleme ve İletişim

- **SpeechRecognition:** Mikrofon dinleme ve ortam gürültüsü kalibrasyonunu yönetir.
- **gTTS (Google Text-to-Speech):** Sistemin durum güncellemelerini ve eksik bilgi taleplerini Neural TTS (Yapay Sinir Ağları tabanlı ses sentezleme) teknolojisiyle doğal bir insan sesi formunda üretir.
- **PyGame:** Üretilen ses dosyalarını ana döngüyü (main thread) kitlemeden, arka planda asenkron olarak oynatan medya yürütücüsü olarak kullanılmıştır.

Sistem Mimarisi (Backend & Concurrency)

- **Python Threading & Queue:** Canlı video akışında ağır matematiksel işlemlerin kamerayı dondurmasını (stuttering) engellemek için, görüntü yakalama ve işlem süreçlerinin eş zamanlı (asenكرون) olarak yürütülmesini sağlar.

3. Sistem Mimarisi ve İletişim Protokolleri

Proje, kullanıcı deneyimini kesintiye uğratmayan çoklu iş parçacıklı bir mimari üzerine inşa edilmiştir:

- **Asenkron Multithreading & Queue:** Gerçek zamanlı video modunda, görüntü işleme algoritmalarının CPU üzerindeki yükü kameranın donmasına sebep olmaz. Bir iş parçacığı (Capture Thread) sürekli veri okurken, işlem kuyruğu (Operation Queue) üzerinden beslenen ikinci iş parçacığı (Processing Thread) efektleri uygular.
- **Non-Destructive (Tahribatsız) İşlem Yığını:** Uygulanan tüm filtreler bir Stack yapısında tutulur. Bu mimari, orijinal görüntü matrisini bozmadan sınırsız Undo ve Reset imkanı sunarken, RAM kullanımını minimize eder.
- Eğer cihazda **CUDA** mevcutsa hesaplamalar **GPU** üzerinden yapılır.

4. Uygulama Modları ve Arayüz

1. **Fotoğraf Modu:** Yerel dosyadaki görseller üzerinde sesli komutlarla düzenleme ve analiz imkanı sunar. Şu anki proje, varsayılan olarak “Lenna” fotoğrafı üzerinden işlemleri gerçekleştirir.
2. **Webcam Modu:** 15 veya 30 FPS seçenekleriyle, canlı kamera akışı üzerinde gerçek zamanlı sinyal işleme ve efekt uygulaması gerçekleştirir. Cihazın kapasitesine göre kalite seçimi yapılabilir.

3. **Heads-Up Display (HUD) Arayüzü:** OpenCV üzerinden render edilen arayüzde; anlık FPS takibi, aktif filtrelerin isimleri ve işlem geçmişi kullanıcıya şeffaf bir panel üzerinden sunulur.

5. Çalışma Mantığı

Sistem 4 temel adımda bir döngü kurar:

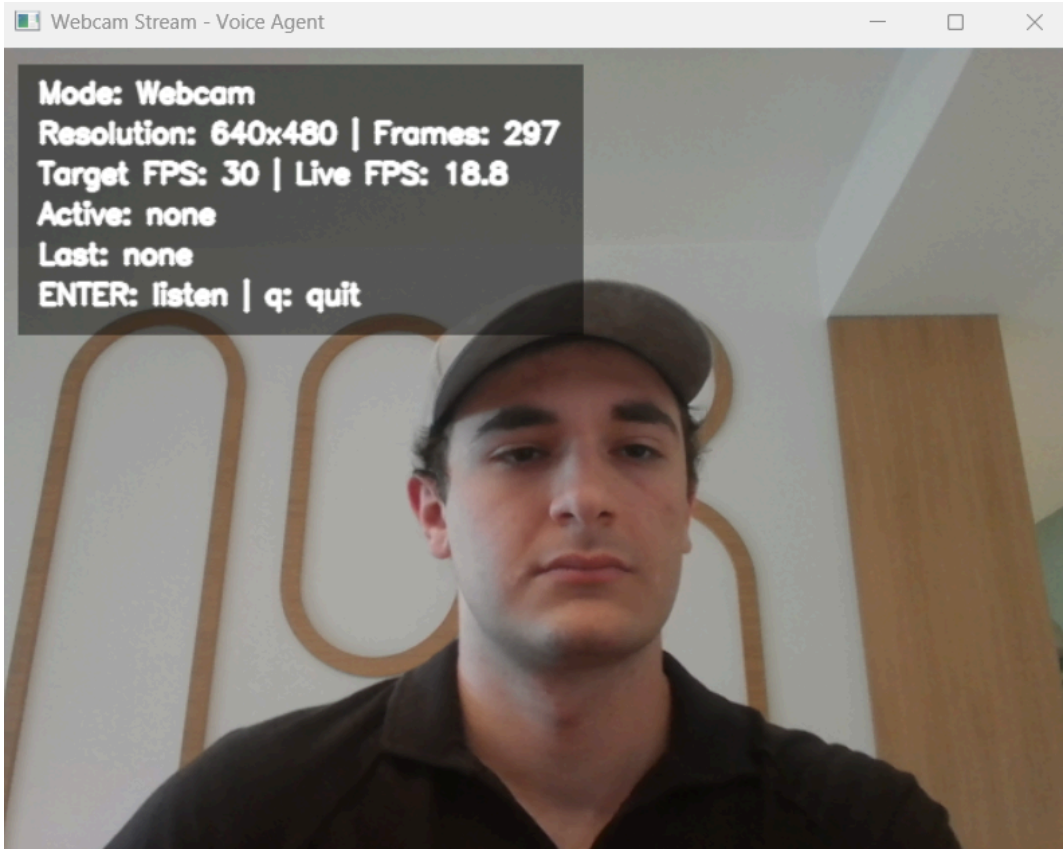
1. **Duyma (Audio Manager):** Mikrofon ortamı dinler, sessizlik olduğunda kaydı keser ve Whisper ile sesi metne döker.
2. **Anlama (Agent Katmanı):** Elde edilen metin yerel Llama 3.1 modeline gönderilir. Model niyet okuması yapar ve sonucu sistemsel bir komuta çevirir (Örn: `{"action": "apply_blur", "intensity": 20}`).
3. **İşleme (Operations):** Gelen JSON komutu backend tarafında yakalanır ve OpenCV/NumPy kullanılarak ilgili matematiksel formüller görüntü matrisine uygulanır.
4. **Gösterme (Video Processor):** İşlenen kareler çoklu iş parçacığı (multithreading) mimarisiyle anında ekrana yansıtılır. İşlemler bir "yığın" (stack) halinde tutulduğu için bellek şişmez.

6. Projeden Ekran Görüntüleri

```
=====
📁 MODE SELECTION - Image Processing Voice Agent
=====
1 Photo Mode    - Process static image with voice commands
2 Webcam Mode   - Process live video stream with voice commands
=====
Select mode (1 or 2): 2
✅ Webcam Mode selected

-----
📺 WEBCAM FPS SELECTION
-----
1 30 FPS - Daha akıcı, biraz daha fazla CPU
2 15 FPS - Daha hafif, daha az CPU
-----
Select FPS (1 or 2): 1
```

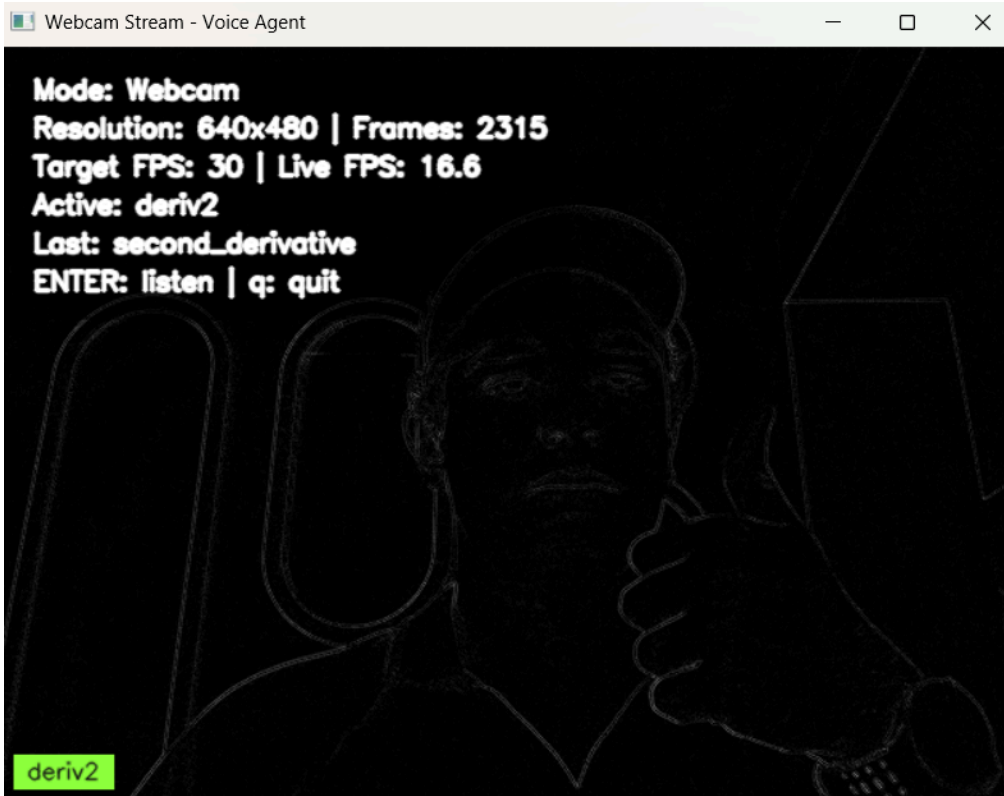
Uygulamayı çalıştırdığımız an seçtiğimiz ayarlar.



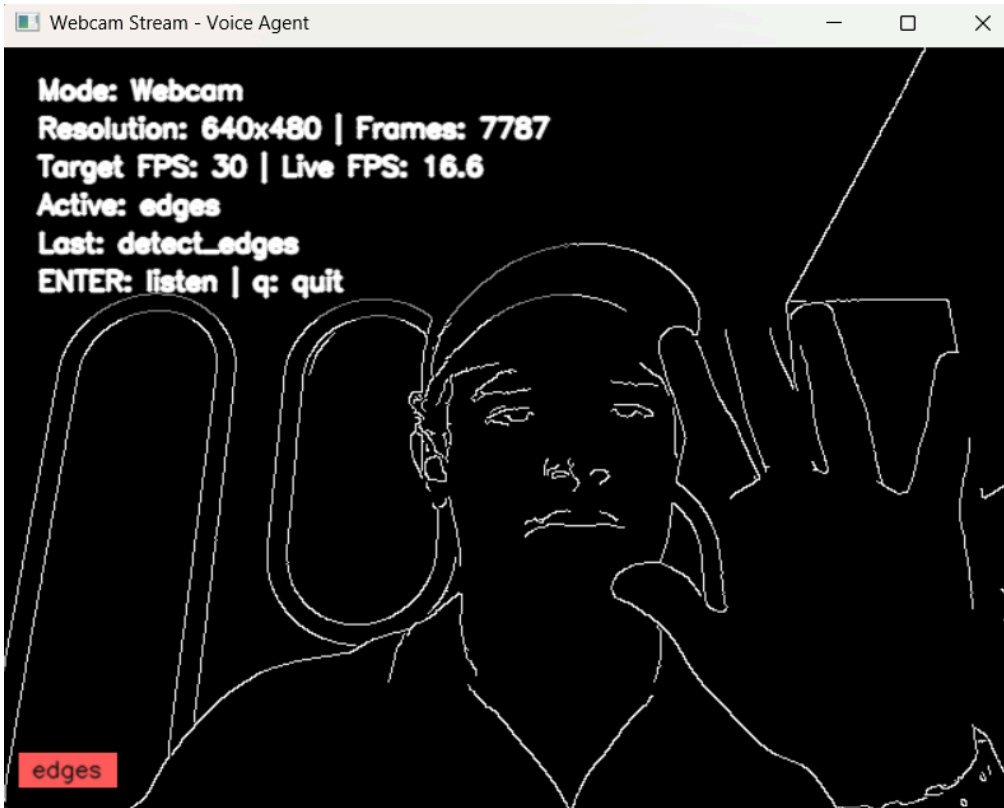
Hiçbir operasyon uygulanmıyorken webcam ekranımız.



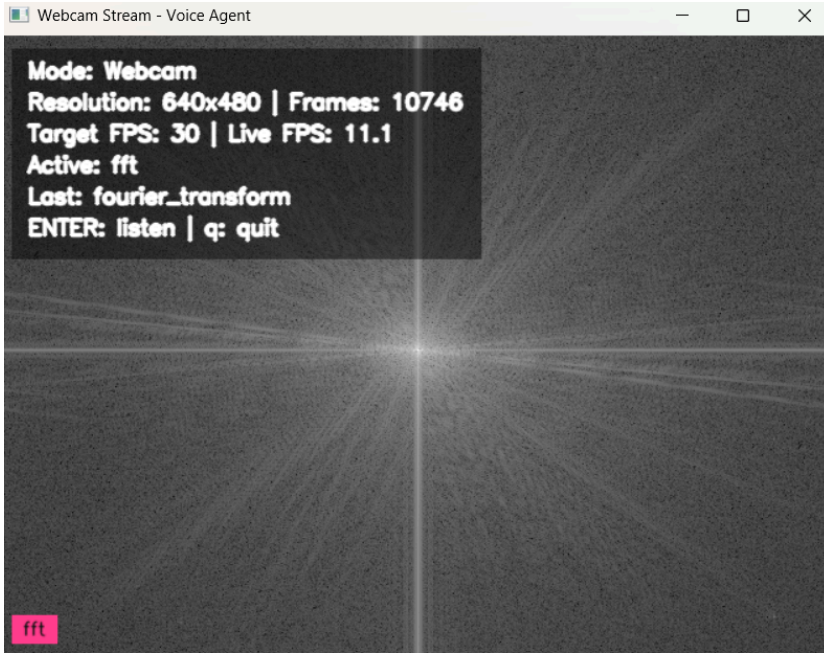
Sadece kırmızı kanalı seçtiğimizde oluşan ekran.



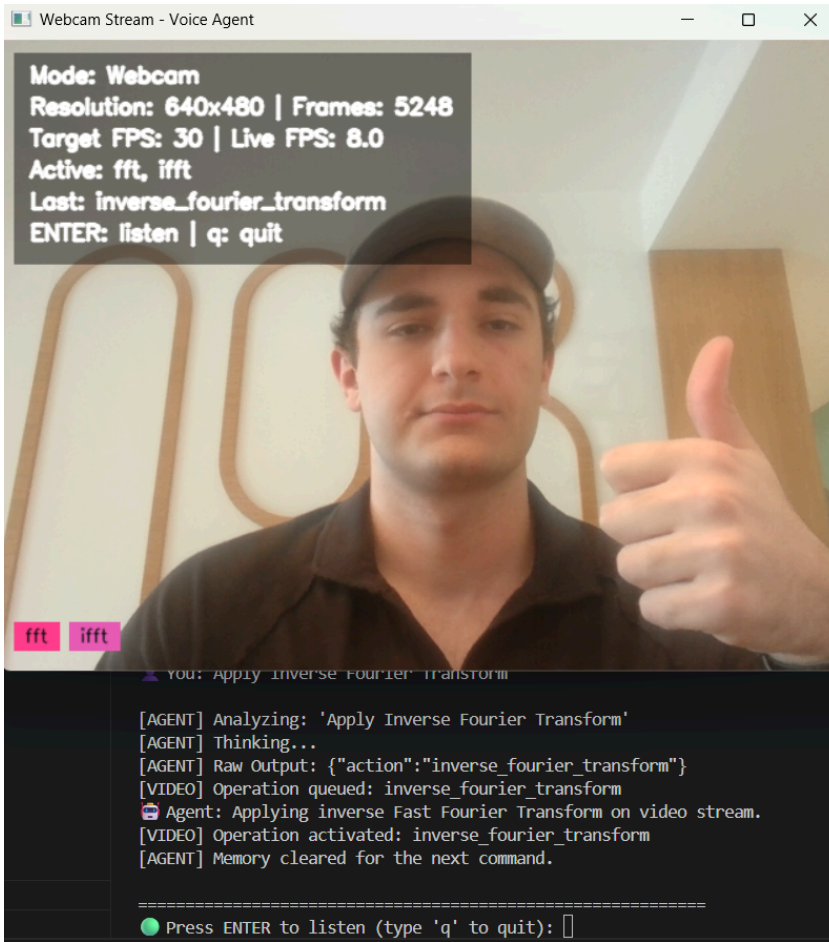
2. Dereceden türev operasyonu uygulandığında oluşan ekran.



Edgeleri göstermesini istediğimizde oluşan ekran.



Webcam e Fourier Transform uygulanmış hali.



Eğer Fourier Transformun üzerine Inverse Fourier Transform komutunu uygularsak, webcam eski haline dönmektedir. Burada Fourier + Inverse Fourier mantığı çalışmaktadır.